

# APP-5258 · Guía de implementación

Qué cambiar y dónde — validación biométrica Facephi Identity V2

Proyecto: `src/micro-person-api` · 10 de agosto de 2026

Este documento es la lista de cambios para ejecutar. El contrato de Facephi, las tablas de códigos y la justificación de cada decisión están en el documento de planeación de APP-5258.

**El código no ha sido compilado.** Se escribió a partir de la revisión del código actual, sin acceso al repositorio. Ajusta *namespaces* y rutas a lo que encuentres.

## Resumen de cambios

| #  | Archivo                                                                   | Acción                 |
|----|---------------------------------------------------------------------------|------------------------|
| 0  | —                                                                         | Verificaciones previas |
| 1  | <code>Common/Enums/FacePhi/ValidationOutcome.cs</code>                    | Nuevo                  |
| 2  | <code>Common/Models/Biometric/ValidateIdentityV2Result.cs</code>          | Nuevo                  |
| 3  | <code>Common/Interfaces/Biometric/IIIdentityValidationEvaluator.cs</code> | Nuevo                  |
| 4  | <code>Services/Biometric/IdentityValidationEvaluator.cs</code>            | Nuevo                  |
| 5  | La clase <code>AppSettings</code> + <code>appsettings.json</code>         | Modificar              |
| 6  | <code>Application/Facephi/Commands/ValidateIdentityV2Cmd.cs</code>        | Modificar              |
| 7  | <code>Endpoints/FacePhi.cs</code>                                         | Modificar              |
| 8  | <code>Program.cs</code>                                                   | Modificar              |
| 9  | <code>Services/Biometric/FacePhiService.cs</code>                         | Modificar              |
| 10 | <code>tests/UnitTests/Application/...</code>                              | Modificar              |

## Principio que guía el diseño

El **cuerpo de la respuesta no cambia**: sigue siendo el JSON que devuelve Facephi, campo por campo. Lo único que se agrega hacia afuera es el **código HTTP**. El veredicto ( `ValidationOutcome` ) es una pieza interna que decide ese código y vive en el handler, donde se puede probar.

Así nadie tiene que preguntar de dónde salió un campo que no está en la documentación de Facephi.

# 0. Verificaciones previas

---

Cuatro comprobaciones de dos minutos cada una. Los cuatro cambian lo que tienes que escribir.

## 0.1 Los valores del enum de liveness

---

```
# abre el archivo y mira si los miembros tienen valores asignados
grep -rn "enum FacephiLivenessResult" -A 25 --include=*.cs src/
```

La regla de decisión hace `(int)result.passiveLivenessResult` y compara contra 3 y 17. Si el enum se declaró **sin valores explícitos**, C# numera desde 0 en orden de declaración y esas comparaciones darían resultados equivocados sin lanzar ningún error.

Debe verse así:

```
public enum FacephiLivenessResult
{
    None = 0,
    Spoof = 1,
    Uncertain = 2,
    Live = 3,
    // ...
    NoLive = 17,
    NoneBecauseEyesClosed = 18
}
```

Si no tiene valores, **corrígelo antes que nada**.

## 0.2 Los headers hacia Facephi

---

```
grep -rn "x-api-key\|DefaultRequestHeaders\|family" --include=*.cs src/
```

Si el flujo de onboarding ya llama a Facephi en producción, la `x-api-key` ya está configurada en algún lado — probablemente en el `AddHttpClient` del `Program.cs`. Lo que sí podría faltar es el header `family`, porque tú eres quien envía el objeto `tracking`.

## 0.3 La llave de configuración de Facephi

---

```
grep -rn "FACEPHI" --include=*.cs --include=*.json src/
```

Confirma que existe `FACEPHI_IDENTITY_API_BASE_URL` y que está poblada en cada ambiente.

## 0.4 Cómo se registra `AppSettings`

---

Abre `Program.cs` y mira si `AppSettings` se registra como singleton, con `IOptions<>` o de otra forma. El evaluador del paso 4 debe inyectarlo **igual que los demás servicios del micro**, no de otra manera.

---

# 1. Nuevo `Common/Enums/FacePhi/ValidationOutcome.cs`

---

Misma carpeta donde vive `FacephiLivenessResult`.

```
namespace onboarding_micro_person.Common.Enums.FacePhi
{
    /// <summary>
    /// Veredicto interno de la validación biométrica. Determina el código HTTP
    /// de la respuesta; no se expone en el cuerpo.
    /// </summary>
    public enum ValidationOutcome
    {
        /// <summary>Rostro POSITIVE, prueba de vida Live y umbral superado.</summary>
        Approved,

        /// <summary>El rostro no coincide, no hay prueba de vida, o no alcanza el umbral.</summary>
        Rejected,

        /// <summary>La captura no permitió evaluar. El cliente debe repetirla.</summary>
        Retry,

        /// <summary>Falló Facephi o la integración.</summary>
        Error
    }
}
```

## 2. Nuevo `Common/Models/Biometric/ValidateIdentityV2Result.cs`

---

Lo que el handler devuelve al endpoint: el veredicto más la respuesta cruda de Facephi. **No se serializa tal cual:** el endpoint publica únicamente la parte `Facephi`.

```
using onboarding_micro_person.Common.Enums.FacePhi;

namespace onboarding_micro_person.Common.Models.Biometric
{
    /// <summary>
    /// Resultado interno del comando: el veredicto decide el código HTTP,
    /// y Facephi es el cuerpo que se devuelve al consumidor sin modificar.
    /// </summary>
    public record ValidateIdentityV2Result(
        ValidationOutcome Outcome,
        PassiveLivenessResult Facephi);
}
```

### 3. Nuevo ▸

Common/Interfaces/Biometric/IIdentityValidationEvaluator.cs

```
namespace onboarding_micro_person.Common.Interfaces.Biometric
{
    /// <summary>
    /// Aplica el umbral de similitud facial definido por el banco para
    /// Identity Validation V2, independiente del que usa onboarding.
    /// </summary>
    public interface IIdentityValidationEvaluator
    {
        bool MeetsSimilarityThreshold(double similarity);
    }
}
```

## 4. Nuevo `Services/Biometric/IdentityValidationEvaluator.cs`

```
using onboarding_micro_person.Common.Interfaces.Biometric;

namespace onboarding_micro_person.Services.Biometric
{
    public class IdentityValidationEvaluator : IIdentityValidationEvaluator
    {
        private readonly AppSettings _appSettings;
        private readonly ILogger<IdentityValidationEvaluator> _logger;

        public IdentityValidationEvaluator(
            AppSettings appSettings,
            ILogger<IdentityValidationEvaluator> logger)
        {
            _appSettings = appSettings;
            _logger = logger;

            // Falla al arrancar, no en la primera validación: si la llave falta,
            // un float sin configurar bindea en 0 y "similitud >= 0" aprueba SIEMPRE.
            var threshold = _appSettings.IDENTITY_VALIDATION_APPROVED_SIMILARITY_VALUE;

            if (threshold is <= 0 or > 1)
            {
                throw new InvalidOperationException(
                    "IDENTITY_VALIDATION_APPROVED_SIMILARITY_VALUE debe estar entre 0 y 1. " +
                    $"Valor actual: {threshold}");
            }
        }

        public bool MeetsSimilarityThreshold(double similarity)
        {
            var threshold = _appSettings.IDENTITY_VALIDATION_APPROVED_SIMILARITY_VALUE;
            var meets = similarity >= threshold;

            _logger.LogInformation(
                "Similitud {Similarity} contra umbral {Threshold}: {Resultado}",
                similarity, threshold, meets ? "supera" : "no supera");

            return meets;
        }
    }
}
```

**Por qué un evaluador propio y no `EvaluateFinalResult`** . El método de onboarding arrastra el bypass que fuerza SUCCESS en QA, solo mira similitud sin saber nada de prueba de vida, y su `FACIAL_VALIDATION_MODE` ( `Average` / `All` ) no aplica: Identity V2 devuelve **una sola** similitud, y sobre un único valor los dos modos son idénticos. Con un evaluador propio, cambiar el umbral de este flujo es tocar un número.

---



## 5. Modificar `AppSettings` y `appsettings.json`

---

### 5.1 La clase de settings

---

Añade la propiedad junto a las demás, respetando la convención del micro:

```
public float IDENTITY_VALIDATION_APPROVED_SIMILARITY_VALUE { get; set; }
```

### 5.2 `appsettings.json` de cada ambiente

---

```
{  
  "IDENTITY_VALIDATION_APPROVED_SIMILARITY_VALUE": 0.75  
}
```

**El valor 0.75 es un ejemplo, no una recomendación.** El número lo define riesgo o fraude, no desarrollo. Mientras se acuerda, usa el mismo que `FACIAL_VALIDATION_APPROVED_SIMILARITY_VALUE` de onboarding y déjalo anotado como pendiente. Subir el umbral reduce fraude pero aumenta los rechazos de clientes legítimos: es un intercambio de negocio.

**No la llames `SOFT_TOKEN_...`** El equipo pidió que nada lleve semántica de Soft Token. El nombre apunta al servicio de Facephi que consumes, no a quién lo usa.

---

## 6. Modificar Application/Facephi/Commands/ValidateIdentityV2Cmd.cs

---

El comando y el validador **no cambian**. Cambia el tipo de retorno, se inyecta el evaluador y se agrega la regla de decisión.

### 6.1 Tipo de retorno del comando

---

```
// ANTES
public class ValidateIdentityV2Cmd : IRequest<PassiveLivenessResult>

// DESPUÉS
public class ValidateIdentityV2Cmd : IRequest<ValidateIdentityV2Result>
```

### 6.2 El handler completo

---

Reemplaza la clase `ValidateIdentityV2CmdHandler` por esta:

```

public class ValidateIdentityV2CmdHandler
    : IRequestHandler<ValidateIdentityV2Cmd, ValidateIdentityV2Result>
{
    // Códigos documentados de Identity Validation V2.
    // facialAuthenticationResult: 0 NONE, 1 NEGATIVE, 3 POSITIVE,
    //                               4 POSE EXCEED, 5 INVALID EXTRACTIONS
    private const int FacialPositive = 3;
    private const int FacialNegative = 1;

    // passiveLivenessResult: 3 Live, 17 NoLive, 1 Spoof (deprecado),
    //                        10 InternalError, 15 LicenseError,
    //                        el resto = no se pudo evaluar
    private const int LivenessLive = 3;
    private const int LivenessNoLive = 17;
    private const int LivenessSpoof = 1;
    private const int LivenessInternalError = 10;
    private const int LivenessLicenseError = 15;

    private readonly IFacePhiService _facePhiService;
    private readonly IIdentityValidationEvaluator _evaluator;
    private readonly ILogger<ValidateIdentityV2CmdHandler> _logger;

    public ValidateIdentityV2CmdHandler(
        IFacePhiService facePhiService,
        IIdentityValidationEvaluator evaluator,
        ILogger<ValidateIdentityV2CmdHandler> logger)
    {
        _facePhiService = facePhiService;
        _evaluator = evaluator;
        _logger = logger;
    }

    public async Task<ValidateIdentityV2Result> Handle(
        ValidateIdentityV2Cmd request,
        CancellationToken cancellationToken)
    {
        _logger.LogInformation(
            "Starting Identity Validation V2. " +
            "Method: {Method}, TrackingProvided: {TrackingProvided}",
            request.Method,
            request.Tracking is not null);

        var result = await _facePhiService.EvaluatePassiveLivenessToken(
            new PassiveLivenessRequest
            {
                Token1 = request.Token1,
                BestImageToken = request.BestImageToken,
            }
        );
    }
}

```

```

        Method = request.Method,
        TrackingToken = request.Tracking?.ExtraData,
        OperationId = request.Tracking?.OperationId
    },
    cancellationToken);

_logger.LogInformation(
    "Identity Validation V2 completed. " +
    "ServiceResultCode={ServiceResultCode}, " +
    "FacialResult={FacialResult}, " +
    "Similarity={Similarity}, " +
    "LivenessResult={LivenessResult}, " +
    "TransactionId={TransactionId}",
    result.ServiceResultCode,
    result.facialAuthenticationResult,
    result.facialAuthenticationSimilarity,
    result.passiveLivenessResult,
    result.ServiceTransactionId);

var outcome = Evaluate(result);

_logger.LogInformation(
    "Identity Validation V2 outcome={Outcome}. TransactionId={TransactionId}",
    outcome,
    result.ServiceTransactionId);

return new ValidateIdentityV2Result(outcome, result);
}

/// <summary>
/// Traduce la respuesta de Facephi a un veredicto.
/// serviceResultCode == 0 sólo indica que el módulo se ejecutó:
/// NO significa que la persona haya sido aprobada.
/// </summary>
private ValidationOutcome Evaluate(PassiveLivenessResult result)
{
    if (result.ServiceResultCode != 0)
    {
        return ValidationOutcome.Error;
    }

    var liveness = (int)result.passiveLivenessResult;

    // Fallos del motor de Facephi: no es culpa de la captura del cliente.
    if (liveness is LivenessInternalError or LivenessLicenseError)
    {
        return ValidationOutcome.Error;
    }
}

```

```

// Rechazos reales: el rostro no coincide o no se detectó vida.
if (result.facialAuthenticationResult == FacialNegative
    || liveness is LivenessNoLive or LivenessSpoof)
{
    return ValidationOutcome.Rejected;
}

// Lista blanca: sólo se evalúa el umbral con POSITIVE y Live explícitos.
// No usar "liveness != NoLive": la doc de evaluatePassiveLiveness
// muestra un rechazo que devuelve 0, no 17.
if (result.facialAuthenticationResult == FacialPositive
    && liveness == LivenessLive)
{
    // Umbral del banco, ADEMÁS del veredicto de Facephi. Nunca en su lugar.
    return _evaluator.MeetsSimilarityThreshold(result.facialAuthenticationSimilarity)
        ? ValidationOutcome.Approved
        : ValidationOutcome.Rejected;
}

// Todo lo demás: la captura no permitió evaluar. El cliente repite.
return ValidationOutcome.Retry;
}
}

```

## 6.3 Usings a añadir

```
using onboarding_micro_person.Common.Enums.FacePhi;
```

## 7. Modificar Endpoints/FacePhi.cs

---

### 7.1 En la cadena del MapPost

---

Añade una línea; el resto se queda igual:

```
.Produces<PassiveLivenessResult>(StatusCodes.Status200OK)
.Produces<PassiveLivenessResult>(StatusCodes.Status422UnprocessableEntity)    // ← nueva
.Produces(StatusCodes.Status400BadRequest)
.Produces(StatusCodes.Status401Unauthorized)
.Produces(StatusCodes.Status403Forbidden)
.Produces(StatusCodes.Status502BadGateway)
.Produces(StatusCodes.Status504GatewayTimeout)
```

## 7.2 El método del endpoint

```
// ANTES
public async Task<IResult> ValidateIdentityV2(
    ValidateIdentityV2Cmd request,
    ISender sender,
    CancellationToken cancellationToken)
{
    var result = await sender.Send(request, cancellationToken);
    return Results.Ok(result);
}

// DESPUÉS
public async Task<IResult> ValidateIdentityV2(
    ValidateIdentityV2Cmd request,
    ISender sender,
    CancellationToken cancellationToken)
{
    var result = await sender.Send(request, cancellationToken);

    // El cuerpo es la respuesta de Facephi sin modificar.
    // Lo único que aporta el micro es el código HTTP: Facephi
    // responde 200 aunque el cliente sea rechazado.
    var statusCode = result.Outcome switch
    {
        ValidationOutcome.Approved => StatusCodes.Status200OK,
        ValidationOutcome.Error => StatusCodes.Status502BadGateway,
        _ => StatusCodes.Status422UnprocessableEntity
    };

    return Results.Json(result.Facephi, statusCode: statusCode);
}
```

## 7.3 Usings a añadir

```
using onboarding_micro_person.Common.Enums.FacePhi;
using onboarding_micro_person.Common.Models.Biometric;
```

# Contrato resultante

| Código    | Cuándo                                               | Qué hace el consumidor                                                                                          |
|-----------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 200       | POSITIVE, Live y umbral superado                     | Continúa con la activación                                                                                      |
| 422       | Rechazo, umbral no alcanzado, o captura no evaluable | Lee <code>facialAuthenticationResult</code> y <code>passiveLivenessResult</code> para decidir si pide recaptura |
| 400       | Falla la validación del request                      | Corrige los datos                                                                                               |
| 502 / 504 | Facephi no disponible o timeout                      | Reintenta                                                                                                       |



## 8. Modificar Program.cs

---

Registra el evaluador junto a los demás servicios, con el mismo ciclo de vida que use `FacePhiService`:

```
builder.Services.AddScoped<IIidentityValidationEvaluator, IdentityValidationEvaluator>();
```

Si `AppSettings` se inyecta con `IOptions<AppSettings>` en el resto del micro, ajusta el constructor del evaluador para que reciba `IOptions<AppSettings>` en vez de la clase directa. Sigue el patrón que ya exista.

---

## 9. Modificar `Services/Biometric/FacePhiService.cs`

Tres cambios dentro de `EvaluatePassiveLivenessToken`. Este archivo es largo; los anclajes indican el punto.

### 9.1 Headers — sólo si el paso 0.2 mostró que faltan

Justo después de construir el `HttpRequestMessage`:

```
var httpRequest = new HttpRequestMessage(HttpMethod.Post, url)
{
    Content = new StringContent(json, Encoding.UTF8, "application/json")
};

// x-api-key es obligatorio. family es requerido al enviar tracking,
// y este payload lo incluye cuando el SDK lo entrega.
httpRequest.Headers.Add("x-api-key", _appSettings.FACEPHI_IDENTITY_API_KEY);
httpRequest.Headers.Add("family", "OnBoarding");
```

### 9.2 No dar por buena una respuesta con error

Después de enviar la petición, antes de leer el cuerpo:

```
if (!response.IsSuccessStatusCode)
{
    // No se loguea el cuerpo: puede contener datos biométricos.
    _logger.LogError(
        "Facephi respondió {StatusCode} en Identity Validation V2.",
        (int)response.StatusCode);

    throw new FacephiIntegrationException(
        $"Facephi respondió con código {(int)response.StatusCode}.",
        (int)response.StatusCode);
}
```

### 9.3 El `catch`

Si el `catch` actual registra el error y devuelve un `PassiveLivenessResult` vacío, ese objeto llega al handler con `ServiceResultCode = 0` y todos los códigos en cero: la regla del paso 6 lo interpretaría como `Retry`. Una caída de Facephi se reportaría al cliente como *"repite la captura"*.

```

catch (FacephiIntegrationException)
{
    throw; // ya tiene el contexto correcto
}
catch (TaskCanceledException ex) when (!cancellationToken.IsCancellationRequested)
{
    throw new FacephiIntegrationException(
        "Timeout invocando Identity Validation V2.", 504, ex);
}
catch (HttpRequestException ex)
{
    throw new FacephiIntegrationException(
        "Error de red invocando Identity Validation V2.", 502, ex);
}

```

Necesitas la excepción, en la carpeta de excepciones que ya use el micro:

```

namespace onboarding_micro_person.Common.Exceptions
{
    public class FacephiIntegrationException : Exception
    {
        public int? UpstreamStatusCode { get; }

        public FacephiIntegrationException(
            string message, int? upstreamStatusCode = null, Exception? innerException = null)
            : base(message, innerException)
        {
            UpstreamStatusCode = upstreamStatusCode;
        }
    }
}

```

Y su traducción a HTTP, en la clase que implementa `IExceptionHandler` (búscala con `grep -rn "IExceptionHandler" --include=*.cs src/`):

```
private async Task HandleFacephiIntegrationException(HttpContext httpContext, Exception ex)
{
    var exception = (FacephiIntegrationException)ex;

    httpContext.Response.StatusCode = exception.UpstreamStatusCode == 504
        ? StatusCodes.Status504GatewayTimeout
        : StatusCodes.Status502BadGateway;

    await httpContext.Response.WriteAsJsonAsync(new ProblemDetails
    {
        Status = httpContext.Response.StatusCode,
        Title = "Error de integración con Facephi",
        Detail = "No fue posible completar la validación biométrica."
    });
}
```

**Cuidado con el `catch` equivocado.** Unas líneas más arriba, en el mismo archivo, hay un `catch` de `FinishTrackingAsync` con un `LogWarning`. **Ese debe seguir tragándose la excepción:** el tracking es accesorio y no debe tumbar la validación. Son dos criterios opuestos conviviendo en el mismo archivo.

## 10. Modificar · tests

---

### 10.1 Renombrar la carpeta

---

```
tests/UnitTests/Application/SoftTokenFacephi/ → tests/UnitTests/Application/Facephi/
```

El namespace de dentro ya dice `...Application.Facephi` : sólo hay que mover la carpeta.

### 10.2 Actualizar el `Setup`

---

El handler ahora recibe tres dependencias:

```
private Mock<IFacePhiService> _facePhiMock = default!;  
private Mock<IIIdentityValidationEvaluator> _evaluatorMock = default!;  
private ValidateIdentityV2CmdHandler _handler = default!;  
  
[SetUp]  
public void Setup()  
{  
    _facePhiMock = new Mock<IFacePhiService>();  
    _evaluatorMock = new Mock<IIIdentityValidationEvaluator>();  
  
    // Por defecto el umbral se supera: cada test que lo necesite lo cambia.  
    _evaluatorMock  
        .Setup(x => x.MeetsSimilarityThreshold(It.IsAny<double>()))  
        .Returns(true);  
  
    _handler = new ValidateIdentityV2CmdHandler(  
        _facePhiMock.Object,  
        _evaluatorMock.Object,  
        Mock.Of<ILogger<ValidateIdentityV2CmdHandler>>());  
}
```

### 10.3 El test del handler que deja de compilar

---

`Handler_MapsRequest_AndReturnsResult` hace assert sobre `result.ServiceResultCode` , que ahora vive dentro de `result.Facephi` :

```
// ANTES
Assert.That(result.ServiceResultCode, Is.EqualTo(0));
Assert.That(result.facialAuthenticationResult, Is.EqualTo(3));

// DESPUÉS
Assert.That(result.Outcome, Is.EqualTo(ValidationOutcome.Approved));
Assert.That(result.Facephi.ServiceResultCode, Is.EqualTo(0));
Assert.That(result.Facephi.facialAuthenticationResult, Is.EqualTo(3));
```

El `_facePhiMock.Verify(...)` que comprueba el mapeo del request **no cambia**.

## 10.4 La regla de decisión

```
[TestCase(3, 3, ValidationOutcome.Approved)] // POSITIVE + Live
[TestCase(1, 3, ValidationOutcome.Rejected)] // NEGATIVE: no es la misma persona
[TestCase(3, 17, ValidationOutcome.Rejected)] // NoLive: sin prueba de vida
[TestCase(3, 0, ValidationOutcome.Retry)] // None: no se pudo evaluar
[TestCase(3, 4, ValidationOutcome.Retry)] // mala calidad de imagen
[TestCase(3, 18, ValidationOutcome.Retry)] // ojos cerrados
[TestCase(0, 3, ValidationOutcome.Retry)] // no se pudo verificar el rostro
[TestCase(3, 10, ValidationOutcome.Error)] // error interno de Facephi
[TestCase(3, 15, ValidationOutcome.Error)] // error de licencia

public async Task Handler_Evalua_El_Verdicto_Segun_Los_Codigos(
    int facialResult, int livenessResult, ValidationOutcome esperado)
{
    _facePhiMock
        .Setup(x => x.EvaluatePassiveLivenessToken(
            It.IsAny<PassiveLivenessRequest>(), It.IsAny<CancellationToken>()))
        .ReturnsAsync(new PassiveLivenessResult
        {
            ServiceResultCode = 0,
            facialAuthenticationResult = facialResult,
            passiveLivenessResult = (FacephiLivenessResult)livenessResult
        });

    var result = await _handler.Handle(
        new ValidateIdentityV2Cmd
        {
            Token1 = "reference-token",
            BestImageToken = "best-image-token",
            Method = "3"
        },
        CancellationToken.None);

    Assert.That(result.Outcome, Is.EqualTo(esperado));
}
```

## 10.5 El umbral

```
[Test]
public async Task Rechaza_Cuando_No_Alcanza_El_Umbral_Aunque_Facephi_Diga_Positive()
{
    _evaluatorMock
        .Setup(x => x.MeetsSimilarityThreshold(It.IsAny<double>()))
        .Returns(false);

    _facePhiMock
        .Setup(x => x.EvaluatePassiveLivenessToken(
            It.IsAny<PassiveLivenessRequest>(), It.IsAny<CancellationToken>()))
        .ReturnsAsync(new PassiveLivenessResult
        {
            ServiceResultCode = 0,
            facialAuthenticationResult = 3,
            facialAuthenticationSimilarity = 0.60,
            passiveLivenessResult = FacephiLivenessResult.Live
        });

    var result = await _handler.Handle(
        new ValidateIdentityV2Cmd
        {
            Token1 = "t",
            BestImageToken = "b",
            Method = "3"
        },
        CancellationToken.None);

    Assert.That(result.Outcome, Is.EqualTo(ValidationOutcome.Rejected));
}
```

Y el evaluador por separado, donde el caso del borde es el que importa:

```
[TestCase(0.99, true)]
[TestCase(0.75, true)]    // el borde exacto: >= aprueba
[TestCase(0.74, false)]

public void Umbral_Se_Aplica_Sobre_La_Similitud(double similarity, bool esperado)
```

## 10.6 Deserialización

El test actual construye el objeto en memoria, así que nunca ejercita el binding del JSON:

```
[Test]
public void Deserializa_La_Respuesta_De_Ejemplo_De_Facephi()
{
    const string json = """
    {
        "serviceTransactionId": "2db602ee-3564-4304-af95-92a52eaae12d",
        "serviceResultCode": 0,
        "serviceResultLog": "[identity] Service executed ok",
        "serviceTime": "2235",
        "facialAuthenticationResult": 3,
        "facialAuthenticationLog": "Positive",
        "facialAuthenticationSimilarity": 0.99214232,
        "passiveLivenessResult": 3,
        "passiveLivenessLog": "Live"
    }
    """;

    var result = JsonSerializer.Deserialize<PassiveLivenessResult>(json!);

    Assert.Multiple(() =>
    {
        Assert.That(result.ServiceResultCode, Is.EqualTo(0));
        Assert.That(result.ServiceTransactionId, Is.Not.Empty);    // detecta binding roto
        Assert.That(result.facialAuthenticationResult, Is.EqualTo(3));
        Assert.That((int)result.passiveLivenessResult, Is.EqualTo(3));
    });
}
```



## 11. Orden y verificación

| Paso | Cambio                                                     | Compila                     |
|------|------------------------------------------------------------|-----------------------------|
| 1    | Verificaciones del paso 0                                  | —                           |
| 2    | ValidationOutcome                                          | Sí                          |
| 3    | FacephiIntegrationException                                | Sí                          |
| 4    | IIdentityValidationEvaluator + IdentityValidationEvaluator | Sí                          |
| 5    | AppSettings + appsettings.json                             | Sí                          |
| 6    | ValidateIdentityV2Result                                   | Sí                          |
| 7    | Handler                                                    | <b>No</b> , hasta el paso 8 |
| 8    | Endpoint                                                   | Sí                          |
| 9    | Program.cs                                                 | Sí                          |
| 10   | FacePhiService                                             | Sí                          |
| 11   | Tests                                                      | Sí                          |

Los pasos 7 y 8 van juntos: cambiar el tipo de retorno rompe el endpoint hasta actualizarlo.

### 11.1 Comandos

```
dotnet build
dotnet test --filter "FullyQualifiedName~ValidateIdentityV2" --logger "console;verbosity=detailed"
```

### 11.2 Pruebas manuales

Con el stub local de Facephi apuntado en `FACEPHI_IDENTITY_API_BASE_URL` :

```
BASE=https://localhost:5001
```

```
# Aprobado → 200
```

```
curl -kX POST $BASE/facephi/validate -H "Content-Type: application/json" \
  -d '{"token1":"ok","bestImageToken":"b","method":"3"}'
```

```
# Rostro no coincide → 422
```

```
curl -kX POST $BASE/facephi/validate -H "Content-Type: application/json" \
  -d '{"token1":"reject","bestImageToken":"b","method":"3"}'
```

```
# Sin prueba de vida → 422
```

```
curl -kX POST $BASE/facephi/validate -H "Content-Type: application/json" \
  -d '{"token1":"nolive","bestImageToken":"b","method":"3"}'
```

```
# Mala calidad → 422
```

```
curl -kX POST $BASE/facephi/validate -H "Content-Type: application/json" \
  -d '{"token1":"retry","bestImageToken":"b","method":"3"}'
```

```
# Facephi caído → 502
```

```
curl -kX POST $BASE/facephi/validate -H "Content-Type: application/json" \
  -d '{"token1":"http502","bestImageToken":"b","method":"3"}'
```

```
# method inválido → 400
```

```
curl -kX POST $BASE/facephi/validate -H "Content-Type: application/json" \
  -d '{"token1":"x","bestImageToken":"y","method":"4"}'
```

## 11.3 Dos verificaciones que no se pueden saltar

### Los logs no deben contener tokens:

```
grep -ri "<el token que usaste en las pruebas>" logs/
# No debe devolver ninguna línea.
```

**No estás probando contra un bypass activo.** `EvaluateFinalResult` de onboarding fuerza SUCCESS cuando el ambiente es QA. Si en algún momento reutilizas ese camino, revisa que en los logs **no** aparezca `BYPASS_ENABLED`; si aparece, la validación no se está ejecutando y la prueba no demuestra nada.

# 12. Decisiones que no son tuyas

Llévalas a tu líder junto con el avance; no las resuelvas solo.

| # | Decisión                                                                                                                           | Quién                     |
|---|------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| 1 | El valor de <code>IDENTITY_VALIDATION_APPROVED_SIMILARITY_VALUE</code>                                                             | Riesgo / Fraude           |
| 2 | ¿El bypass de QA debe aplicar al flujo que entrega el segundo factor? ¿Qué impide que quede en <code>SUCCESS</code> en producción? | Líder técnico / Seguridad |
| 3 | ¿ <code>/facephi/validate</code> lleva autenticación o queda interno tras el gateway?                                              | Arquitectura              |
| 4 | Cuántos reintentos ante <code>Retry</code> antes de bloquear la activación                                                         | Producto / Riesgo         |
| 5 | ¿Se requiere validar la autenticidad del documento? Identity V2 compara caras, no verifica que la cédula sea legítima              | Producto / Riesgo         |

Guía de implementación de APP-5258. El código no ha sido compilado: requiere ajuste de namespaces y validación contra el repositorio real de `onboarding-micro-person` .